Square Of Zeroes

AlgoExpert · Very Hard · Dynamic Programming

The problem

- Does the 0/1 matrix contain a square (side >= 2) whose border is entirely 0s?
- Precompute consecutive zeros to the right and downward per cell.

Intuition

- Precompute zeros to the right and below each cell, so each square border is checked in constant time.

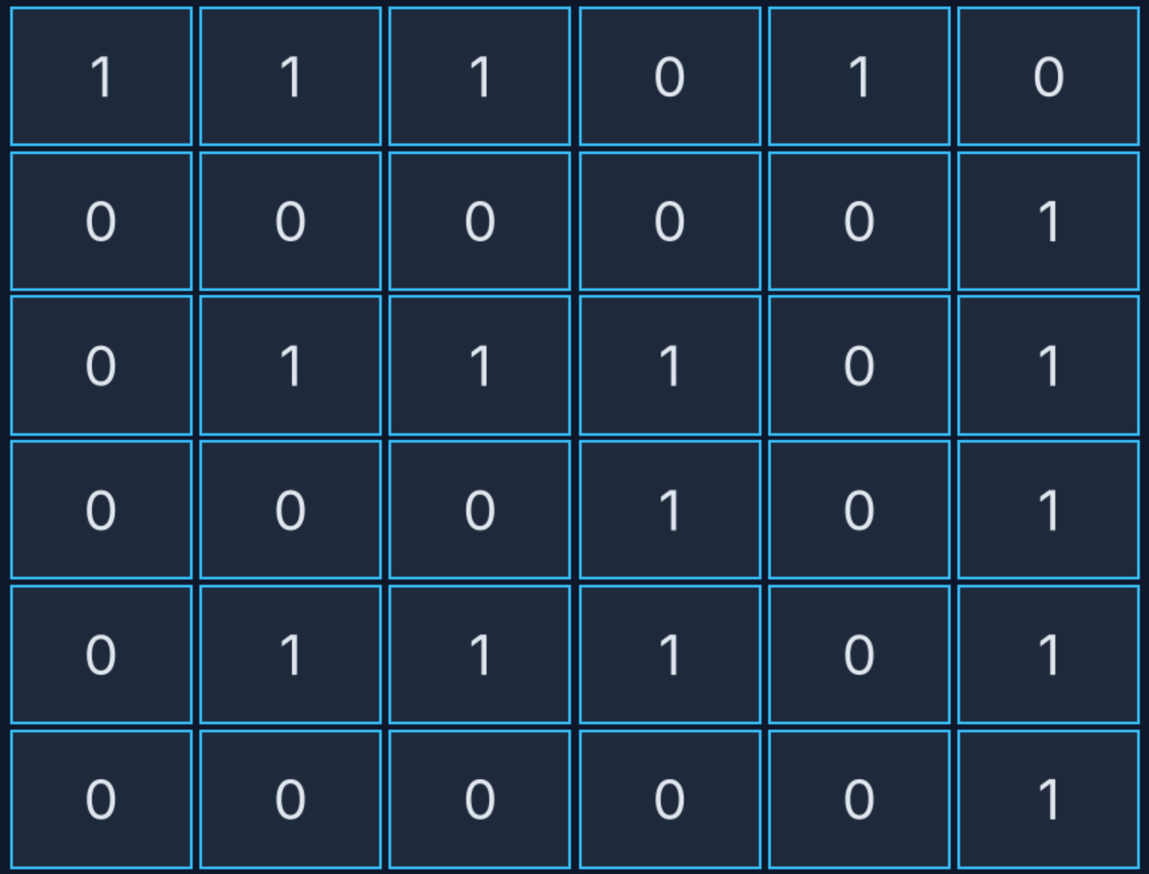
Brute force vs optimal

- Check every square's border cell by cell → $O(n^4)$
- Precomputed runs → $O(n^3)$
- $O(1)$ side checks

Approach

- 1. Build right and down zero counts
- 2. For each top-left and size
- 3. Check the four sides with the counts
- 4. Return true on success

Worked example



Find a square whose border is all 0s

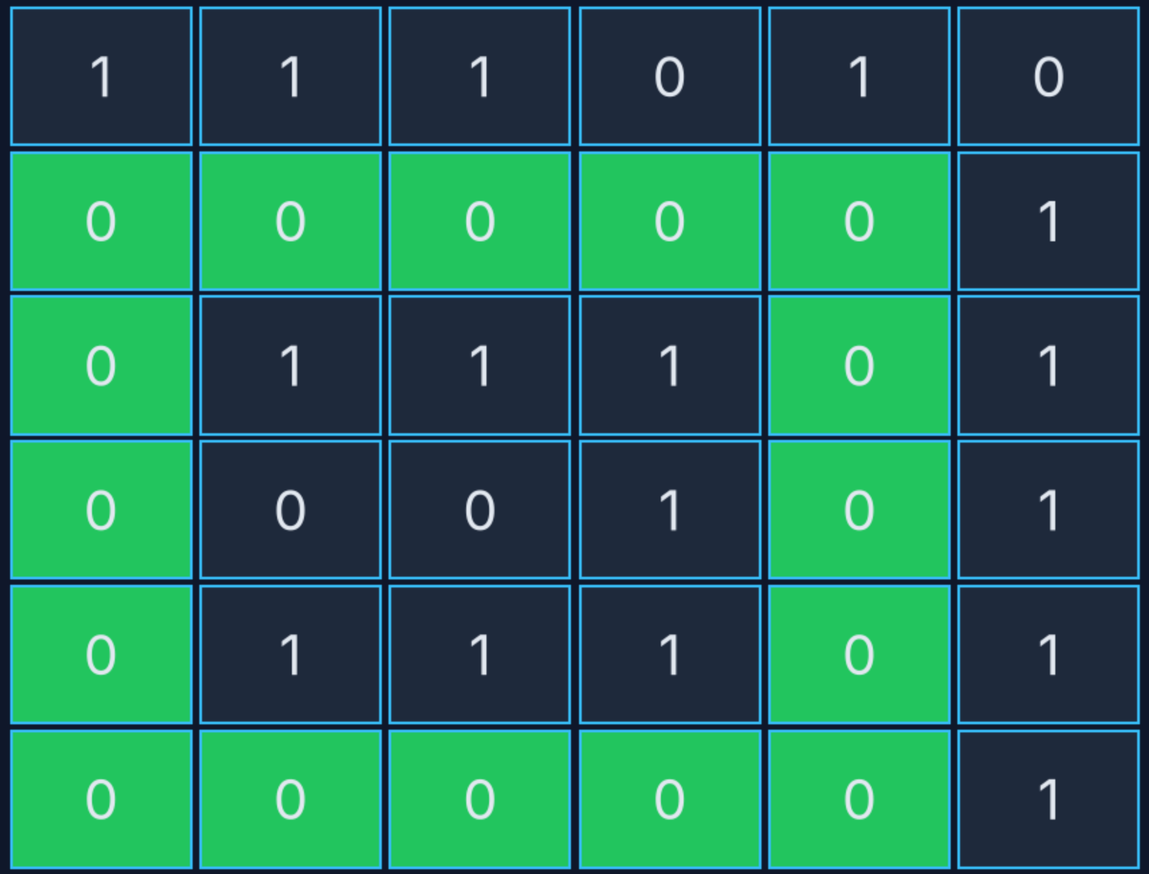
Worked example

```
precompute right[r][c]: zeros to the right

and down[r][c]: zeros downward
```

Precompute zero runs

Worked example



Check corners in O(1) per square

Java solution

```java
public class SquareOfZeroes {
    public static boolean squareOfZeroes(int[][] m) {
        int n = m.length;
        int[][] right = new int[n + 1][n + 1], down = new int[n + 1][n + 1];
        for (int r = n - 1; r >= 0; r--) for (int c = n - 1; c >= 0; c--) if (m[r][c] == 0) {
            right[r][c] = right[r][c + 1] + 1;
            down[r][c] = down[r + 1][c] + 1;
        }
        for (int r = 0; r < n; r++) for (int c = 0; c < n; c++)
            for (int s = 2; r + s <= n && c + s <= n; s++)
                if (right[r][c] >= s && down[r][c] >= s && right[r + s - 1][c] >= s && down[r][c + s - 1] >= s) return true;
        return false;
    }
}
```

Complexity

- Time: O(n^3)
- Space: O(n^2)

Edge cases & pitfalls

- Size ≥ 2
- 1×1 doesn't count
- Pad arrays by one

Interview tips & follow-ups

- Explain the four checks

Recap

- Precompute zeros to the right and below each cell, so each square border is checked in constant time.
- Complexity: O(n^3)